

Brute Force, Divide-and-Conquer & Dynamic Programming

Answer all parts of this question. Show all calculations and intermediate steps where applicable. You may use a non-programmable calculator.

1. A logistics company has developed a route planning system for its delivery vehicles. The current system examines **every possible route** before selecting the shortest one. Initially, the system performs well, but its execution time increases dramatically as the number of delivery locations grows.
 - a. Identify the algorithm design strategy currently used by the system and explain how it works in the given scenario.
 - b. Assume the company has **8 delivery locations**.
 - i. Calculate the total number of possible routes that must be evaluated using the current approach.
 - ii. If the system can evaluate **2 million routes per second**, estimate the total execution time required.

Show all calculations.

- c. State **two advantages** and **two disadvantages** of the current algorithm design strategy in the given scenario.
- d. Recommend a more suitable algorithm design strategy to improve the system performance. Justify your recommendation.

2. Following your recommendation in **Question 1**, the development team decides to redesign several modules of the system using the **Divide and Conquer** strategy.
- A. Using a suitable diagram, explain the three phases of the Divide and Conquer strategy.
 - B. The company needs to sort the following delivery priorities before assigning vehicles. : [42, 15, 67, 28, 91, 53, 36, 74]
 - C. Using the **Merge Sort** algorithm, demonstrate how the above array is sorted by clearly showing the:
 - Divide phase
 - Recursive subproblems
 - Merge process
 - Final sorted array
 - D. The same dataset is now sorted using the **Quick Sort** algorithm, where the **last element is always selected as the pivot**. Construct the complete Quick Sort recursion tree and show every partitioning step until the array becomes fully sorted.
 - E. The Merge Sort algorithm recursively divides an array until each subarray contains only one element.
 - I. For the given array containing 8 elements, determine the number of levels in the recursion tree. Show your calculation.
 - II. Determine the total number of subarrays created during the Divide phase. Show your calculation.
 - III. During the Merge phase, how many merge operations are required to obtain the final sorted array? Show your calculation.

3. After improving the sorting module, the company introduces an image-processing module that repeatedly multiplies large matrices to analyse traffic patterns collected from road cameras.

- a. Multiply the following matrices using the **conventional matrix multiplication** method.

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 2 \\ 6 & 1 \end{bmatrix}$$

Show all calculations.

- b. Explain how **Strassen's Matrix Multiplication** reduces the number of multiplication operations compared with the conventional matrix multiplication method.
- c. Illustrate your answer by identifying the seven intermediate products (P1-P7)
- d. Compare the **conventional matrix multiplication** algorithm and **Strassen's Matrix Multiplication** with respect to the following aspects:
- Divide and Conquer strategy
 - Number of multiplication operations
 - Time complexity
 - Practical advantages and limitations

4. Although the company's sorting and matrix multiplication modules have been improved, its warehouse robot planning system still performs poorly because it repeatedly solves the same recursive subproblems. A robot can move either one rack or two racks at a time.

Let $R(n)$ denote the number of different ways to reach the n^{th} rack.

$$R(n) = R(n-1) + R(n-2)$$

$$\text{where: } R(1)=1, R(2)=2$$

- A. Explain why Dynamic Programming is more suitable than Divide and Conquer for this problem. In your answer, discuss independent subproblems, overlapping subproblems, and optimal substructure.
- B. Draw the recursive computation tree for **R(6)** and identify the repeated subproblems.
- C. Calculate $R(8)$ using the **Memoization** (Top-Down) approach. Show the recursive calls and the final memoization table.
- D. Calculate **R(8)** using the **Tabulation** (Bottom-Up) approach. Show the completed Dynamic Programming table. Hence, state which approach is more suitable for this problem.